

P, NP, NP-Complete, and NP-Hard

Decision Problems, Verification, Reductions, and Complexity Classes

1 The Big Picture

Central Question

Complexity theory studies how much computational resource is needed to solve a problem. In this topic, the main resource is **time**, and the main boundary is **polynomial time**.

The most important distinction is between:

- **solving** a problem: finding the correct answer from scratch;
- **verifying** a solution: checking a proposed certificate or witness.

Most definitions in this topic are stated for **decision problems**.

Decision Problem

A decision problem is a problem whose output is only YES or NO.

Decision vs. Optimization

Optimization Version	Decision Version
Find the shortest path from s to t.	Is there a path from s to t of length at most k ?
Find the minimum TSP tour.	Is there a TSP tour of cost at most k ?
Find the largest clique.	Does the graph contain a clique of size at least k ?

2 Polynomial Time

Polynomial-Time Algorithm

An algorithm runs in **polynomial time** if its running time is bounded by

$$\mathcal{O}(n^c)$$

for some constant c , where n is the input length.

Polynomial time is the standard theoretical meaning of “efficiently solvable.” For example, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$, and $\mathcal{O}(n^5)$ are polynomial-time bounds. In contrast, $\mathcal{O}(2^n)$ and $\mathcal{O}(n!)$ are not polynomial-time bounds.

3 Class P

Definition of P

$\mathbf{P} = \{\text{decision problems solvable in polynomial time}\}.$

A problem is in $\mathbf{P}$ if there is a deterministic algorithm that always returns the correct YES or No answer in polynomial time.

Examples in P

- Is an array sorted?
- Is a graph connected?
- Is there a path from s to t in a graph?
- Is there a shortest path from s to t with length at most k ?
- Is an integer n prime?

Key Intuition

If a problem is in $\mathbf{P}$, then we can solve it efficiently. Since solving gives the answer directly, verifying a proposed answer is also efficient.

4 Class NP

Definition of NP

$\mathbf{NP} = \{\text{decision problems whose YES certificates can be verified in polynomial time}\}.$

For a problem in $\mathbf{NP}$, a YES answer has a short proof called a **certificate** or **witness**. A polynomial-time verifier checks that certificate.

Verifier View of NP

A decision problem L is in $\mathbf{NP}$ if there exists a polynomial-time algorithm V and a polynomial p such that:

$$x \in L \iff \exists y, |y| \leq p(|x|) \text{ and } V(x, y) = \text{YES}.$$

Here x is the input instance and y is the certificate.

Problem	Certificate	Verification Idea
SAT	Truth assignment	Substitute values and check that the formula is true.
3-SAT	Truth assignment	Check every 3-literal clause.
Clique	Set of k vertices	Check that every pair in the set has an edge.
Vertex-Cover	Set of k vertices	Check that every edge has at least one endpoint in the set.
Hamiltonian-Cycle	Ordering of vertices	Check that consecutive vertices are connected and all vertices appear once.

Relation Between P and NP

$$\mathbf{P} \subseteq \mathbf{NP}.$$

Every problem that can be solved quickly can also be verified quickly.

Open Problem

It is unknown whether

$$\mathbf{P} = \mathbf{NP}$$

or

$$\mathbf{P} \neq \mathbf{NP}.$$

This is the famous **P versus NP** problem.

5 Polynomial-Time Reductions

Why Reductions Matter

Reductions let us compare problem difficulty. To prove that problem B is at least as hard as problem A , we show how to convert any instance of A into an instance of B in polynomial time.

Polynomial-Time Many-One Reduction

For decision problems A and B , we write

$$A \leq_p B$$

if there is a polynomial-time computable function f such that, for every input x ,

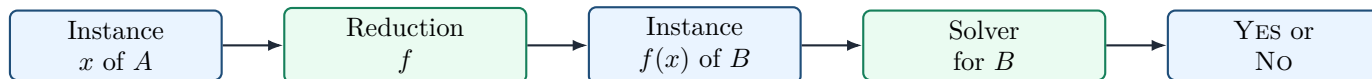
$$x \in A \iff f(x) \in B.$$

In words, $A \leq_p B$ means:

- every instance of A can be translated into an instance of B ;
- the translation is efficient;
- the answer is preserved.

Direction of Reduction

If $A \leq_p B$, then B is at least as hard as A . A fast algorithm for B would also give a fast algorithm for A .



6 NP-Hard

Definition of NP-hard

A problem H is **NP-hard** if every problem in **NP** can be polynomial-time reduced to H :

$$\forall L \in \mathbf{NP}, \quad L \leq_p H.$$

NP-hard means “at least as hard as every problem in **NP**.” An **NP-hard** problem does not have to be a decision problem, and it does not have to belong to **NP**.

Important Facts About NP-hard

- **NP-hard** problems are at least as hard as all problems in **NP**.
- They may be optimization problems instead of decision problems.
- They may not have efficiently verifiable certificates.
- They may even be undecidable in broader complexity settings.

Examples of NP-hard Problems

- TSP optimization version: find the minimum-cost tour.
- Knapsack optimization version: find the maximum value under capacity.
- Halting problem: **NP-hard** but not in **NP** because it is undecidable.

7 NP-Complete

Definition of NP-complete

A decision problem C is **NP-complete** if:

1. $C \in \mathbf{NP}$;
2. C is **NP-hard**.

NP-complete problems are the hardest problems inside **NP**. They are special because they are both verifiable in polynomial time and at least as hard as every problem in **NP**.

Consequence

If any **NP**-complete problem has a polynomial-time algorithm, then every problem in **NP** has a polynomial-time algorithm. Therefore:

If one **NP**-complete problem is in **P**, then **P** = **NP**.

Examples of NP-complete Problems

- SAT
- 3-SAT
- CLIQUE decision version
- VERTEX-COVER decision version
- HAMILTONIAN-CYCLE
- TSP decision version

Common Exam Mistake

TSP has two common forms:

- Decision version: “Is there a tour of cost at most k ?” This is **NP**-complete.
- Optimization version: “Find the minimum-cost tour.” This is **NP**-hard.

8 How to Prove a Problem Is NP-Complete

Standard Proof Template

To prove that a problem B is **NP**-complete:

1. Prove $B \in \mathbf{NP}$ by giving a polynomial-time verifier.
2. Choose a known **NP**-complete problem A .
3. Prove $A \leq_p B$ using a polynomial-time reduction.
4. Show the reduction preserves the answer:

$$x \in A \iff f(x) \in B.$$

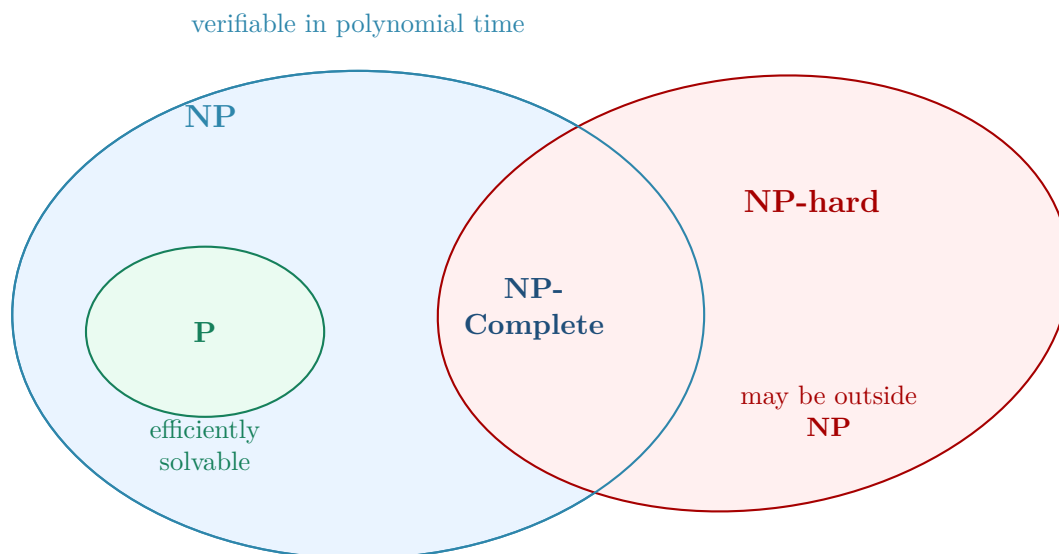
Reduction Direction

To prove B is **NP**-hard, reduce a known hard problem **to** B :

$$\text{known } \mathbf{NP}\text{-complete problem } A \leq_p B.$$

Reducing B to a known **NP**-complete problem does not prove that B is hard.

9 Relationship Diagram



How to Read the Diagram

- **P** is inside **NP** because fast solving implies fast verification.
- **NP-complete** is the overlap between **NP** and **NP-hard**.
- Some **NP-hard** problems are outside **NP**.
- Whether **P** equals **NP** is unknown.

10 Comparison Table

Class	Main Meaning	Typical Example
P	Can be solved in polynomial time.	Graph connectivity, shortest-path decision problem.
NP	A YES certificate can be verified in polynomial time.	SAT, CLIQUE, VERTEX-COVER.
NP-hard	At least as hard as every problem in NP . May not be in NP .	TSP optimization, knapsack optimization.
NP-complete	Both in NP and NP-hard . Hardest problems within NP .	SAT, 3-SAT, HAMILTONIAN-CYCLE, TSP decision version.

11 Memory Guide

Quick Rules

- **P**: problems we can solve quickly.
- **NP**: problems whose YES answers can be checked quickly.
- **NP-hard**: problems at least as hard as all problems in **NP**.
- **NP-complete**: problems that are both in **NP** and **NP-hard**.

Most Important Formulas

$$\mathbf{P} \subseteq \mathbf{NP}$$

$$\mathbf{NP-complete} = \mathbf{NP} \cap \mathbf{NP-hard}$$

If any **NP-complete** problem is in **P**, then $\mathbf{P} = \mathbf{NP}$.

12 Quick Self-Check

Review Questions

1. What is the difference between solving and verifying?
2. Why is every problem in **P** also in **NP**?
3. What is a certificate?
4. What does $A \leq_p B$ mean?
5. If $A \leq_p B$, which problem is at least as hard?
6. What two things must be shown to prove a problem is **NP-complete**?
7. Why is the TSP decision version **NP-complete** but the optimization version **NP-hard**?
8. What would happen if one **NP-complete** problem were solved in polynomial time?